Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech - ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-2027	Semester:	V
Name of the Student:	P.Sohith	Roll Number:	A24126552170
Name of the Lab:	FS	Timings:	

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (10 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4Marks)	Programming Knowledge (4 Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	1	3	4	5	1	13	10	
8/7	2	3	4	5	1	13	10	
15/7	3	3	4	4	1	12	10	
22/7	4	3	4	4	2	13	10	
29/7	5	3	4	4	1	12	10	
5/8	6	3	4	4	2	13	10	
12/8	7	3	4	5	1	13	10	
19/8	8	3	4	5	1	13	10	
2/9	9	3	4	5	1	13	10	
9/9	10	3	4	5	1	13	10	

Name of the Faculty:

Faculty Signature:

Introduction to HTML and CSS.

→ HTML (Hyper text Markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (cascading style sheets) is a stylesheet language used to control the Presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions, and layouts. CSS also supports responsive design, allowing webpages to adopt to different screen sizes and devices. CSS can be applied using inline, internal or external style sheets.

→ HTML and CSS work ... webpages. where HTML provides the struc and content. while CSS provides th layout, visual Presentation. The combi of both forms th foundation of modern development and enables the creation of organized, attractive, and user-frien web interfaces.

Difference between MERN Stack and Full

→ MERN Stack is a javascript - based technology Stack used for developing moder applications. MERN stands for Mongo DB, Express.js, React.js, Node.js, MongoDB is for database management, Express.js handles the backend framework and API React.js is used to develop frontend use interface, and Node.js Provides the ser side runtime environment.

→ Full-Stack Development refers the develo -ment of both the frontend and backend a web application along with databa integration. A full stack developer work with technologies used for designing the use interface, implementing server-side logic

HTML components:

1. HTML tags: Define the structure and elements of a webpage.
2. Headings - used to display titles and headings using <h1> to <h6>
3. Paragraphs - used to display text using the <p> tag
4. Links - used to navigate between webpages using <a> tag.
5. Images - used to display images using <image> tag
6. Lists - used to create ordered and unordered lists using <ol>, <ul>,
7. Tables - used to organize data in rows and columns.
8. Forms - used to collect user input using elements such as text boxes, buttons and checkboxes
9. Div and span - used to group and organize webpage content
10. Semantic elements - Elements such as <header>, <nav>, <selection>, <article> <footer>. Provide meaningful webpage structure.

CSS components:

1. Selectors: Select HTML elements to apply sty
2. Properties - Define the visual characteristic of elements
3. values - specify the setting of a CSS Prope
4. colors - Define text, background, border co
5. Fonts - control font family, size, style an weight
6. Backgrounds - set background colors and i
7. Borders - Define the border style, width and radius
8. Margin - controls the space outside an element
9. Padding - controls the space inside elem
10. layout - controls the arrangement of elements using Flexbox, Grid and Positioning.

HTML VS HTML5

HTML	HTML5
older version of HTML	Latest major version of HTML supports audio and vedio
limited multimedia support	
uses external Plugins for multimedia	No external plugs needed
Has fewer semantic tags	Provides semantic tags
No <canvas> element	supports <canvas>
Limited web storage	supports local storage
complex DOCTYPE declaration.	simple <!DOCTYPE html>

CSS VS CSS3

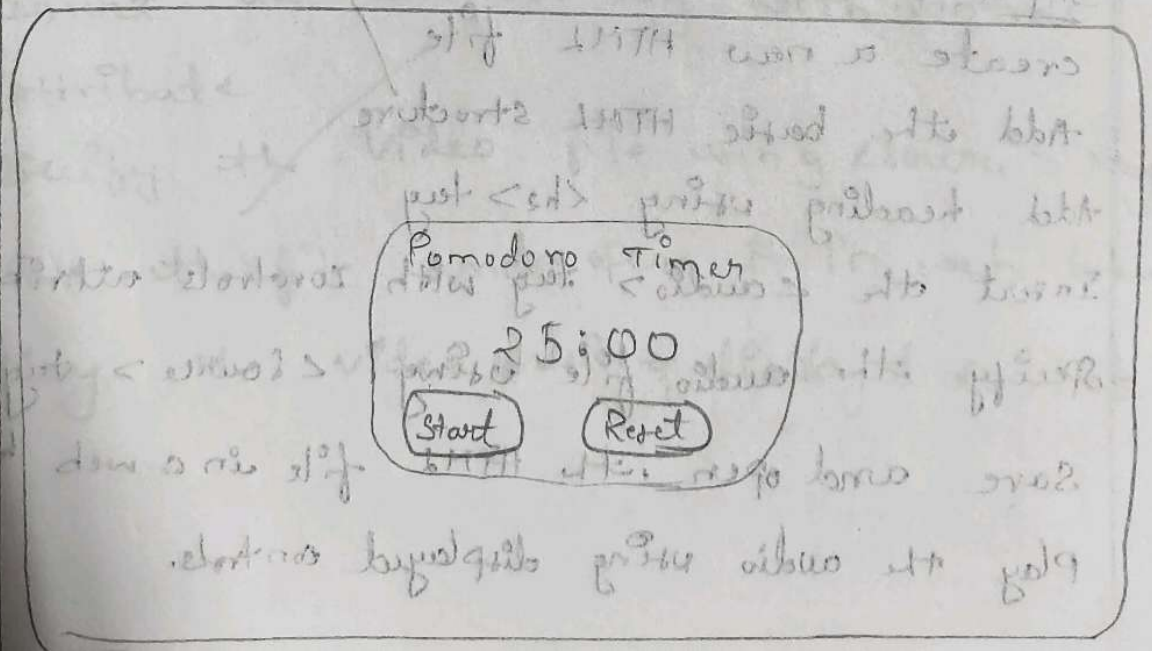
CSS	CSS3
Earlier version of CSS	Advanced version of CSS
Basic styling features	Provides advanced styling features.
Limited animations	supports animations and transitions.
No flexible layout systems	Supports flexbox and grid
Limited visual effects	supports shadows, gradients, effects
No media queries	supports media queries
Uses a single specification	Divided into different

Design a static web page using HTML and CSS
Pomodoro Timer, using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
        body{
            font-family: Arial;
            background: #f5f5f5;
            text-align: center;
            Padding: 50 Px;
        }
        .timer{
            width: 300Px;
            margin: auto;
            Padding: 30 Px;
            background: white;
            bader-radius: 15 Px;
            box-shadow: 0 0 10Px gray;
        }
        h1{
            color: #e74c3c;
        }
        .time{
            font-size: 50 Px;
            margin: 20Px 0;
        }
        button{
            Padding: 10Px 20Px;
            margin: 5Px;
```

```
        border-radius: 5px;
        background: #e74c3c;
        color: white;
    }
</style>
</head>
<body>
    <div class="timer">
        <h1>Pomodoro Timer</h1>
        <div class="time">25:00</div>
        <button>Start</button>
        <button>Reset</button>
    </div>
</body>
</html>
```

O/P:



1) Design an HTML code using audio tags.

code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Audio Example</title>
</head>
<body>
    <h2> my favourite song </h2>
    <audio controls>
        <source src="song.mp3"
        type="audio/mpeg">
        Your browser does not support
        the audio tag
    </audio>
</body>
</html>
```

O/P: Displays an audio player with play, Pause and volume controls.

Steps:

create a new HTML file
Add the basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with controls attri
Specify the audio file using <source> ta
Save and open the HTML file in a web
Play the audio using displayed controls.

ii) Design a [illegible] using video tags:

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> video Example </title>
</head>
<body>
    <h2> my vedio </h2>
    <vedio width = "400" controls>
        <source src = "video.mp4" type = "video/mp4">
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and full screen controls.

Steps:

create a new HTML file

Add a basic HTML structure

Add a heading using the <h2> tag

Insert the <video> tag with .controls attribute

specify the video file using <source> tag

save the file and open it in web brows

play the video using displayed controls.

ii) Design a HTML code using, vedio tags:

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> my vedio </h2>
    <video width = "400" controls>
        <source src = "video.mp4" type = "video/mp4">
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and full screen controls.

Steps:

create a new HTML file

Add a basic HTML structure

Add a heading using the <h2> tag

Insert the <video> tag with .controls attribute

specify the video file using <source> tag

Save the file and open it in web browser

play the video using displayed controls.

write a Program using Javascript (arrays, functions)

code:

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];

Function displayarray(arr){
    console.log("Array Elements:");
    for(let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}
function findsum(arr){
    let sum=0;
    for(let i=0; i<arr.length; i++
        sum+=arr[i];
    }
    return sum;
}
```

EXPERIMENT - 04

A. Develop dynamic webpage using java script

Program:

```
<!DOCTYPE html>
<html>
<head>
<title> Dynamic webpage </title>
<style>
body{
    text-align : centre ;
    font-family : Arial;
    margin-top : 100 Px ;
}
button {
    Padding : 10 Px 20 Px ;
    font-size : 16 Px ;
    cursor : Pointer ;
}
</style>
</head>
<body>
<h1 id = "heading"> welcome to my
webpage </h1>
<p id = "message"> click the button to change
this webpage </p>
```

```
<button onclick="changePage()">click me</button>

<script>
    function changePage(){
    document.getElement by Id ("heading")
    innerHTML="welcome to javascript
    document. getelement by Id ("heading")
        innerHTML="welcome to javascript!
    document.getElementById("message")
        innerHTML="the webpage has been
    changed dynamically";
    document.body.style.backgroundcolor=
                "lightblue";
    }
</script>
</body>
</html>
```

Welcome to my webpage

click the button to change the webpag

[clickme]

• When the user clicks "clickme", the javascrip function changepage () is called, then the heading dynamically changes and backgrou also changes.

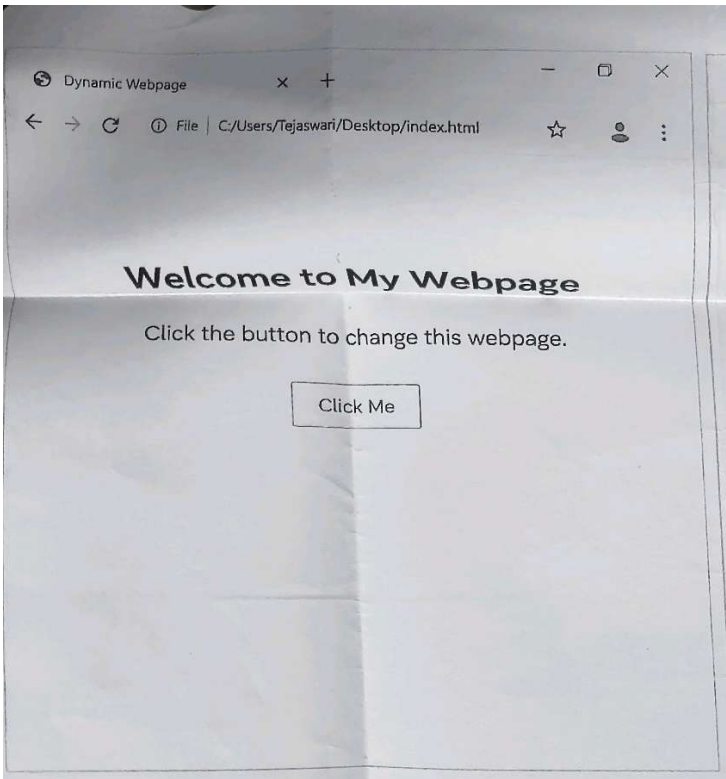
After clicking the button

welcome to javascript !

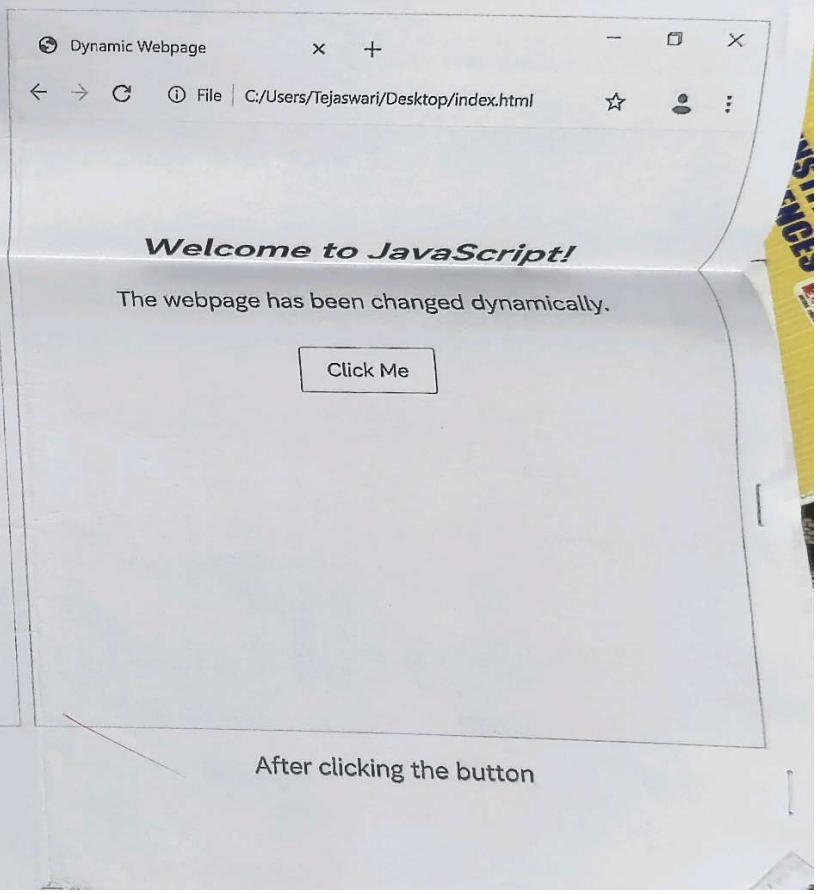
The webpage has been changed dynamically

[click me]





Before clicking the button



After clicking the button

3. Using Built-in Core modules:

Nodejs also comes equipped with dozens of built-in modules (like fs for file systems or path for file routing) that don't need installation.

We will load them by module names:

```
// common js style
const fs = require('fs');
// ES modules style
import fs from 'fs';
```

Here are some of the most commonly used ones:

1. fs = file system operations
2. http = HTTP server and client
3. path - file path utilities
4. os - operating system utilities
5. events - Event handling
6. util - utility functions
7. stream - stream handling
8. crypto - cryptographic functions
9. url - URL parsing
10. query string - URL query string handling

Example: Simple HTTP server

```
http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end("Hello World!");
}).listen(8080);
```

output: Hello World!

6) Develop mini application using express and Node.js.

- Simple mini application using node.js + Express.js

Student Management system:

1. Add a student
2. View all students
3. delete a student
4. Use express.js router
5. Simple HTML interface

Project structure :-

```
student-app/
├── server.js
└── Public
    └── index.html
```

server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("Public"));
app.get("/students", (req, res) => {
    res.json(students);
});
```

```
//-Add
app.post("/students", (req, res) => {
    const student = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course
    };
    students.push(student);
    res.json({ message: "Student added successfully",
    student: student });
});

// Delete
app.delete("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    students = students.filter(student =>
        student.id !== id);
    res.json({
        message: "Student deleted successfully"
// Start server
app.listen(PORT, () => {
    console.log("Server running at http://
```

Public/index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Management
</head>
<body>
    <h1> Student management system
    <input type="text" id="nam
    Placeholder=" Enter student na
  <input type="text" id="cour
    Placeholder=" Enter course ">
  <button onclick="add student(
    Add student </button>
    <h2> Student list </h2>
    <ul id="studentlist"> </ul>
<script>
    function add student(){
    let name=document.getElement
                Id("name").val
      let course =document.get Elem
                ById("course").va
```

```
fetch("/students", {
    method: "POST",
    headers: {
        "content-Type": "applic
    },
    body: JSON.stringify({
        name: name,
        course: course
    })
})
.then(response => response.json
.then(data => {
    alert(data.message
    document.getElementById("name").value
    document.getElementById("course").value
    displayStudents();
})
}
function displayStudents() {
    fetch("/students").then(respo
        response.json().th
        (students => {
    let list = document.getElementById(
```

```
students.forEach(student => {
    const li =
    document.createElement("li");
    li.innerHTML = student.id + "-" +
                   student.name + "-" +
                   student.course + "<button
onclick='deleteStudent(" + student.id + ")'>
                   Delete</button>";
    list.appendChild(li);
  });
}

function deleteStudent(id) {
  fetch("/students/" + id, {
    method: "DELETE"
  })
  .then(response => response.json())
  .then(data => {
    alert(data.message);
    displayStudents();
  });
}

displayStudents();
</script>
</body>
</html>
```

output :

Student management system

Enter student name

Enter course

Add Student

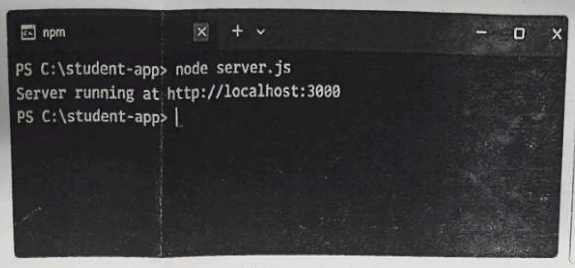
Student List :-

1. Manohar – CSE Delete
2. Manoj – CSE Delete

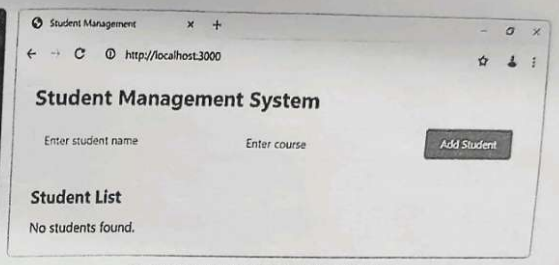
the Express.js server is successfully sta

(at http://localhost:3000

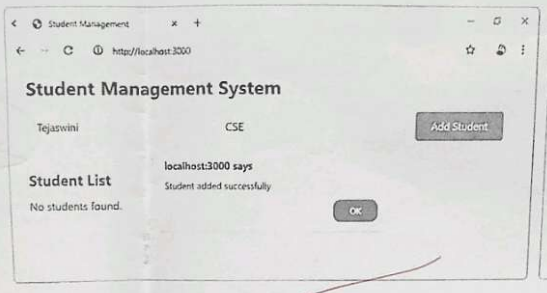
the browser displays the student management system with fields



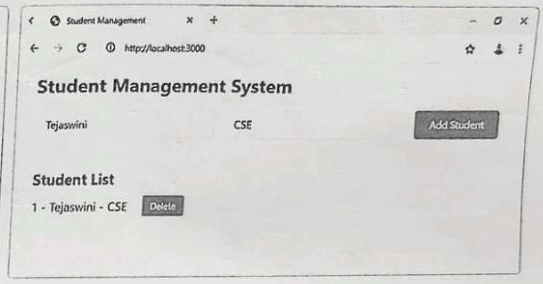
1. Terminal Output (after running `node server.js`)



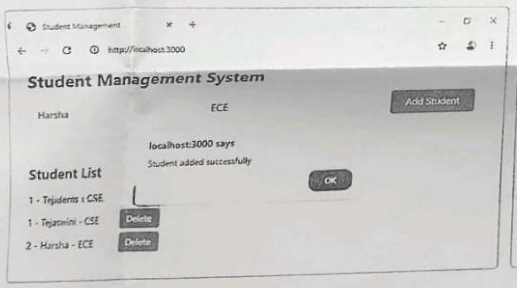
2. Initial Output in Browser (no students)



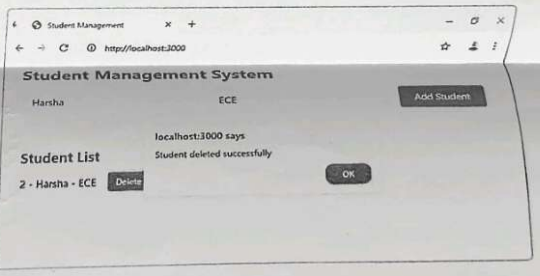
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

EXPERIMENT -7

7. Create a Functional component and class component using react.

Aim: To develop a react application using Functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage.

• Functional component:

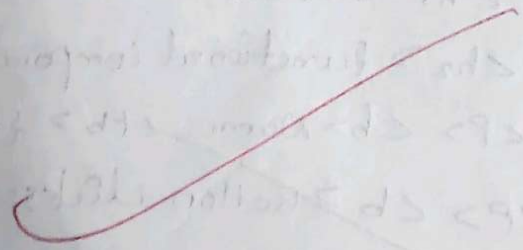
A Functional component is a javascript function that returns JSX to display the UI. Interface. It uses react Hooks such as useState to manage state and update the webpage dynamically.

Program:

```
import react, {useState} from
'react':
function studentfunctional(){
    const [name, setName] = useState("Ashok");
    const [cause, setcourse] = usestate("CSM");
    const [count, setcount] = useState(0);
    const changeName = () => {
        setName("Ashokk");
    };
```

```
const changeCourse = () => {
    setCourse("CSD");
};
const incrementCount = () => {
    setCount(count++);
};
return (
  <div style={{
        border: "8px solid blue",
        Padding: "20Px";
        margin: "20Px,
        borderRadius: "10Px",
        textAlign: "center"
    }}>
    <h1>Student details</h1>
    <h2>Functional Component</h2>
    <P> <b>Name:</b> {name}</P>
    <P> <b>Button clicks:</b>{count}</P>
    <button onClick={changeName}>
        changeName</button>
    <button onClick={changeCourse}>
        change course</button>
    <button onClick={incrementCount}>
        Increment</button>
  </div>
```

output: When the user the buttons, the name course and count change dynamically without reloading the webpage.

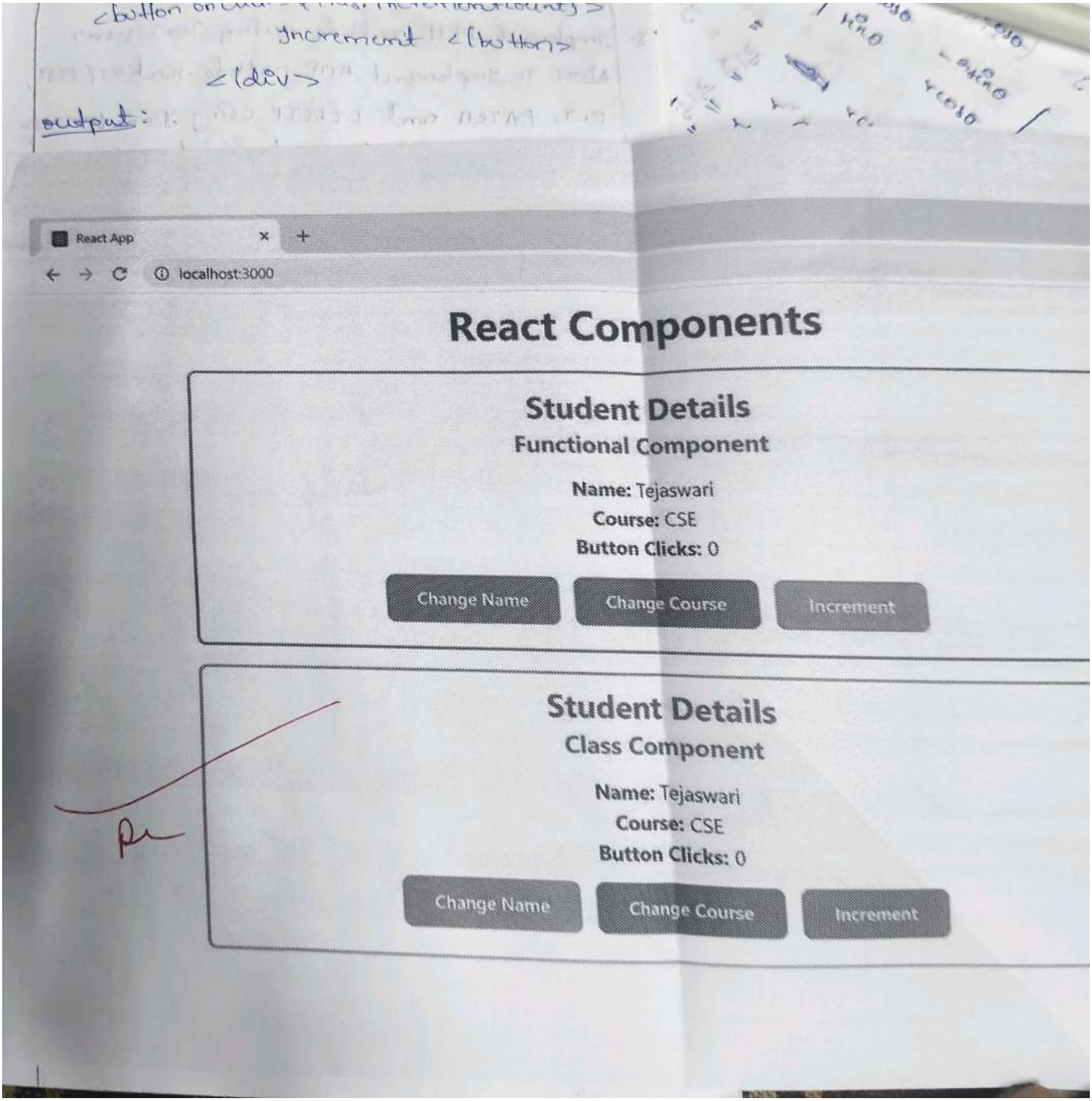


.class component:

A class component is an ES6 JavaScript class that extends React.component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program:

```
import React {Component} from
'react';
class Studentclass extends Component {
    constructor (Props){
        super (Props);
        this.state = {
            name = "Ashok",
            course = "computer science"
            count = 0
        };
    }
    changeName = () => {
        this.setState({
            name: "Ashokk"
        });
    };
    changeCourse = () => {
        this.setState({
            course: "CSM"
        });
```


Increment </button>

output:
React App
localhost:3000
React Components
Student Details
Functional Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment
Student Details
Class Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment

```
Increment
    this.setState((PrevState)=>{
        count: PrevState.count+1
    }));
}
render(){
    return(
        <div style={{
            border: "2Px solid green",
            Padding: "20Px",
            margin: "20Px",
            borderRadius: "10Px",
            textalign: "center"
        }}>
        <h1> Student details </h1>
        <h2> Class Component </h2>
        <P><b>Name: </b>{this.state.name}</P>
        <P><b>Course: {this.state.course}</P>
        <P><b>Button clicks: </b>{this.state.co
        <button onClick={this.changeName}>
            changeName</button>
        <button onClick={this.changeCourse}>
            change course</button>
```

EXPERIMENT - 08

8. Implement HTTP methods using JSON server

Aim: To implement HTTP methods such as (POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create, Read, Update, Delete) operations on student records.

Theory: JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch. HTTP methods are used to communicate between a client and a server.

- GET - Retrieves data from the server
- POST - Adds new data to the server
- PUT - Replaces an existing record
- PATCH - updates selected fields of a record
- DELETE - Removes a record the server

a. Create a JSON server

Step1: Initialize the Project

open the VS code terminal and execute the following commands.

```
mkdir JSON server
cd JSON server
npm init -y
npm install JSON server
```

Step2: create db.json

create a file named db.json and add the following student data.

```
{
  "Students": [
    {
      "id": "1",
      "name: "Ashok",
      "course": "CSE",
      "mark": 90
    },
    {
      "id": "2",
      "name": "AB",
      "course": "CSE",
      "marks": 90
    },
    {
      "id": "3"
      "name": "BC",
      "course": "IT",
      "marks": 88
    }
```

Step 3: Start the JSON server

For the current JSON server version, use

npx json-server db.json

The server start at:

http://localhost:3000

The Student records can be accessed

http://localhost:3000/students

2. Implementation of HTTP methods

a. GET Method: The GET method is used to read student records from JSON server

Code:

```
fetch('http://localhost:3000/students')
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

output:

B. POST Method : The POST method is used to insert a new student record into the server.

code :

```
fetch ('http://localhost:3000/students', {
    method: 'POST',
    headers {
    'content-type': 'application/json'
    },
    body. JSON. stringify ({
        name: 'Rahul';
        course: 'CSE'
        marks : 92
    })
})
.then(response => response.json())
.then(data =>{
        console. log(data);
})
```

```
.catch (error => {
    console.log(error);
});
```

output:

c. PUT Method: the PUT method is used to repla
on existing student record with update da

code:
```
fetch ('http://localhost:3000/students/
method : 'PUT',
headers : {
    'content-type' : 'application/json,
},
body : JSON.stringify({
    name : 'Ashok',
    course : 'AIML',
    marks : 95
})
})
.then (response => response.json())
```

```
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

output:

D. PATCH Method: The PATCH method is used to update only selected fields of an existing student record.

code:

```
fetch('http://localhost:3000/students/2', {
    method: 'PATCH',
    headers: {
        'Content-Type': 'application/json',
    },
    body: JSON.stringify({
        marks: 96
    })
})
.then(response => response.json())
.then(data => {
```

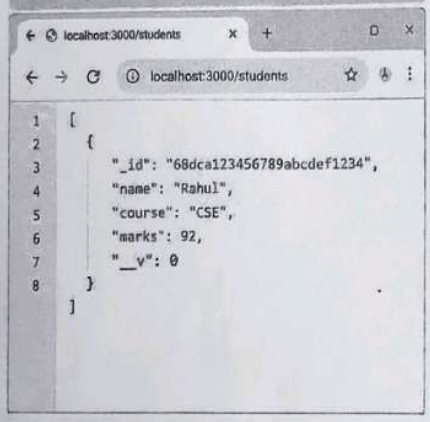
```
console.log(allData);
```

9)

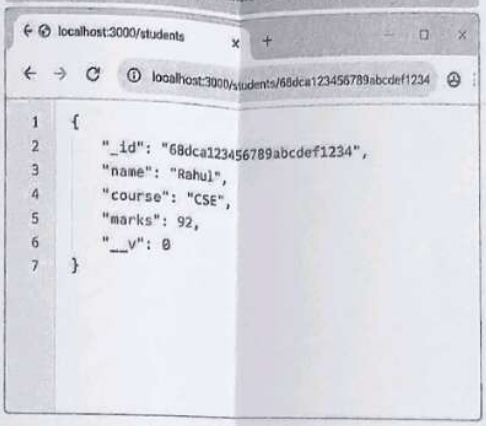
```
.catch(err => {
  console.log(err);
```

output:

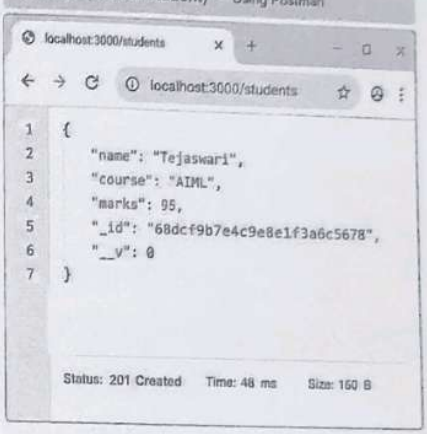
1. GET All Students (http://localhost:3000/students)



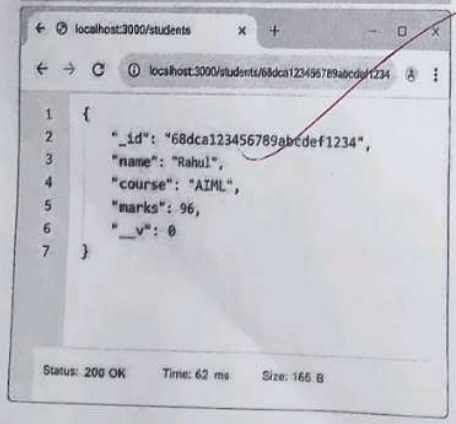
2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)



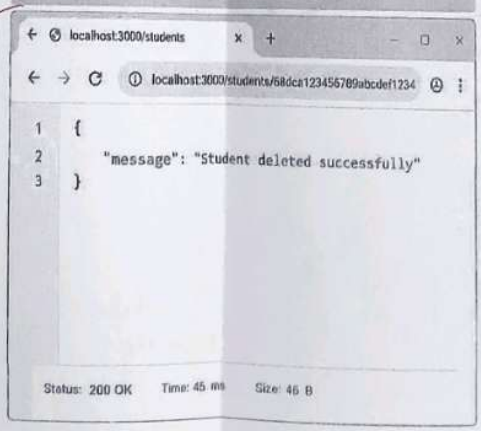
3. POST (Create Student) – Using Postman



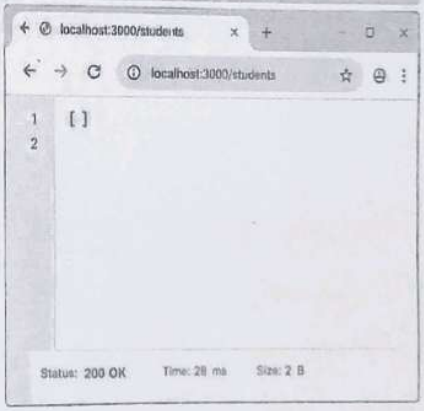
4. PUT (Update Student) – Using Postman



5. DELETE (Remove Student) – Using Postman



6. GET All Students (After Deletion)



3)

requests.

output:

-After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining Student will be:

output:

-Thus, HTTP methods GET, POST, PUT, PATCH and DELETE were successfully implemented using JSON server the CRUD operations were performed on Student records the data has retrieved, inserted, updated and deleted using HTTP requests

1. Develop CRUD operations using Mongo DB

Aim: To develop a web application using Node.js Express.js and Mongo DB to perform CRUD operations (Create, Read, Update, Delete) on Student records.

Theory: Mongo DB is a NoSQL database that stores data in flexible, JSON-like documents. CRUD operations are used to manage data in a database.

Mongoose in an Object Data Modeling (ODM) library for Mongo DB and Node.js. It helps define schemas, interact with collection and perform database operations.

1. Installation and Project setup:

```
Mkdir MongoDB-CRUD
cd MongoDB-CRUD
npm init -y
npm install express mongoose
```

Program: server.js

```
const express. require('express');
const mongoose. require('mongoose');
const app = express();
const PORT = 3000;
app.use (express.json());

  mongoose.connect('mongodb://127.0.0.1:27017/
                                    college')
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.log(err));
// create.student
const Student schema = new
mongoose.Schema({
      name: {
        type: String
        required: true
    },
    course: {
        type: String,
        required: true
    },
    marks: {
        type = Number
        required: true
```

```
});
});
// Create Student model
const Student =
    mongoose.model('Student', StudentSch

// CREATE: Add
app.post('/Students', async (req, res)=
    try{
        const student = new Student(req.b
        const saved Student = await Student
                                    savel

    res.status(201).json(save Student);
    } catch (error) {
        res.status(400).json({ message:
            error.message });
        }
    });

    // READ: Get all Students
    app.get('/Students', async (req, r

        try {
            const Student = await
```

```
Student.find():
res.status(500).json({message: error.message});
}
});
// READ: Get student by ID
app.get('/students/:id', async(req, res)=>{
  try{
    const student = await
    Student.findById(req.params.id);
    if(!student){
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json(student)
  }catch(error){
    res.status(500).json({message:
      error.message});
  }
});
// update
app.Put('/student/:id',async(req,res)=>{
  try{
```

```
const student = await
student findByAndUpdate(
    req.Params.id,
    req.body
    {new: true, runValidators: true}
);
if (!student){
    return res.Status(404).json({
     message: "student not found"
    })
}
res.Status(200).json(student);
} catch (error){
    res.status(400).json({message
     (error.message})
}
});
//DELETE
app.delete('/students/:id', async(req,res)=>
    try{
     const student await student
                findBYID Delete(
```

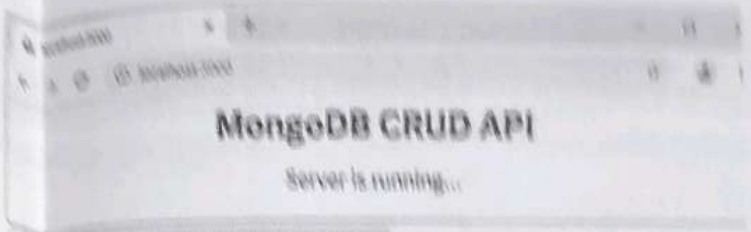
```
    req.Params.id
  );
  if (!student) {
    return res.status(404).json({
      message: "Student not found"
    });
  }
  res.status(200).json({
    message: "Student deleted
              successfully"
  });
} catch (error) {
  res.status(500).json({ message:
    error.message });
}
});
```

output:

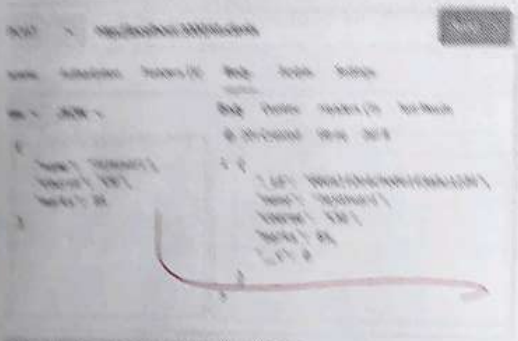




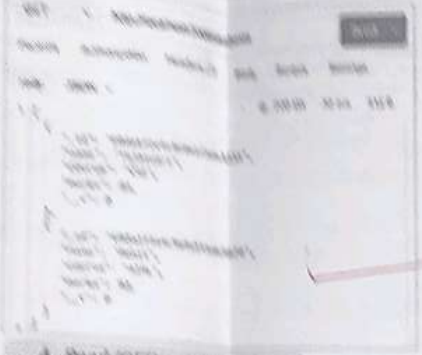
1. Terminal Output



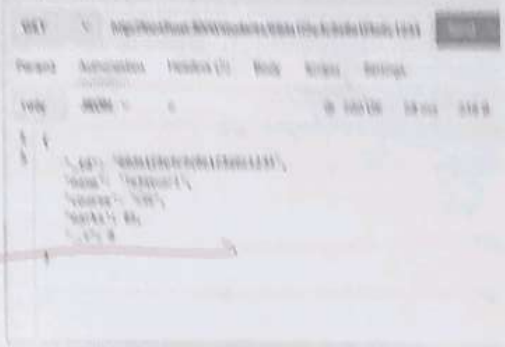
2. Chrome Browser (Home Page)



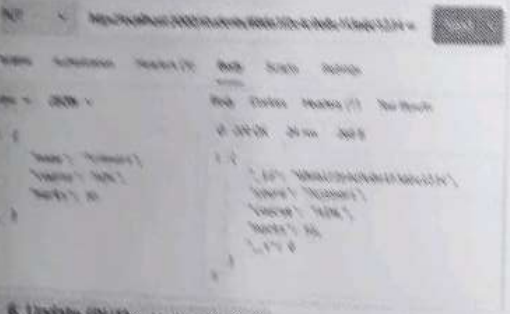
3. Create (POST) – Add Student



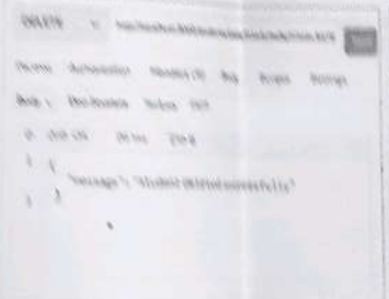
4. Read (GET) – Get All Students



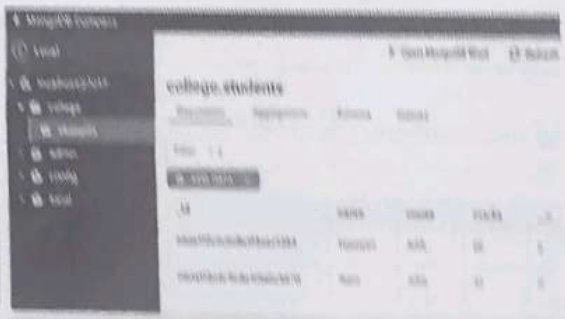
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View